Deep Learning Practicals 2–6: Aim, Theory and Conclusion

Simple notes you can use for your journal and viva. Every practical has the same parts: Aim, Theory, Steps in short, and Conclusion.

Basic Terms Used in All Practicals

- **Neuron:** a small unit that takes numbers in, multiplies them by weights, adds them up and gives one number out.
- **Layer:** a group of neurons. A network has an input layer, hidden layers and an output layer.
- **Weights:** the numbers the network learns. Training means finding good weights.
- **Activation function:** a rule applied to a neuron's output so the network can learn complex patterns. Examples are ReLU and softmax.
- **Loss function:** tells how wrong the model's prediction is. Lower is better.
- **Optimizer:** the method that changes the weights to reduce the loss (SGD, Adam).
- **Epoch:** one full pass over the whole training data.
- **Batch size:** how many samples are used before the weights are updated once.
- **Validation data:** a part of the training data kept aside to check the model while it trains.
- **Overfitting:** the model does very well on training data but badly on new data, because it memorized instead of learning.
- **Normalization:** scaling input values to a small range such as 0 to 1 so training is faster and more stable.

Practical 2: Feed-Forward Neural Network (MNIST Digit Recognition)

Aim

To build, train and evaluate a feed-forward neural network using TensorFlow and Keras that recognizes handwritten digits (0 to 9) from the MNIST dataset.

Theory

Artificial Neural Network (ANN). An ANN is a computer model inspired by the human brain. It is made of layers of connected neurons that learn from examples.

Feed-Forward Neural Network (FFNN). In an FFNN, information moves in one direction only: from the input layer, through the hidden layers, to the output layer. There are no loops. Each neuron in one layer is connected to every neuron in the next layer, so these layers are called Dense (fully connected) layers.

How a neural network learns.

1. **Forward pass:** the input goes through the layers and the network makes a prediction.
2. **Loss calculation:** the prediction is compared with the true answer and the error (loss) is calculated.
3. **Backpropagation:** the error is sent backwards through the network to find how much each weight contributed to it.
4. **Gradient descent:** each weight is changed a little in the direction that reduces the error.

These four steps repeat for every batch, for many epochs, until the loss is small.

Dataset: MNIST. It has 60,000 training images and 10,000 test images of handwritten digits. Each image is 28 × 28 pixels in grayscale, and each pixel value is between 0 and 255.

Preprocessing used.

- **Normalization:** dividing pixels by 255 gives values from 0 to 1.
- **One-hot encoding:** the label 3 becomes [0,0,0,1,0,0,0,0,0,0]. This matches the 10 outputs of the network.

Model used.

- **Flatten:** turns the 28 × 28 image into a line of 784 numbers.
- **Dense(128, ReLU) and Dense(64, ReLU):** hidden layers that learn patterns in the digits.
- **Dense(10, Softmax):** output layer that gives 10 probabilities adding up to 1. The largest probability is the predicted digit.

Important functions.

- **ReLU:** gives the number itself if it is positive, otherwise 0. It adds non-linearity and trains quickly.
- **Softmax:** converts the output scores into probabilities.
- **Categorical cross-entropy:** the loss for multi-class problems with one-hot labels. It becomes large when the model is confident about a wrong class.
- **SGD (Stochastic Gradient Descent):** the optimizer that updates weights using small batches of data.

Evaluation. The model is tested on the 10,000 test images it has never seen. The accuracy and loss graphs of training and validation show how well it learned.

Steps in Short

1. Load MNIST.
2. Normalize the images and one-hot encode the labels.
3. Build the model: Flatten, Dense 128, Dense 64, Dense 10.
4. Compile with SGD, categorical cross-entropy and accuracy.
5. Train for 5 epochs with batch size 128 and a 10% validation split.
6. Evaluate on test data and plot accuracy and loss.

Conclusion

A feed-forward neural network was built and trained on the MNIST dataset. It learned to recognize handwritten digits with about 90 percent test accuracy. The accuracy graph rose and the loss graph fell over the epochs, which shows that the network learned properly. The training and validation curves stayed close, so there was no serious overfitting. Accuracy can be improved by using the Adam optimizer, more epochs, or more layers.

Practical 3: Image Classification Using a CNN (CIFAR-10)

Aim

To build an image classification model with a Convolutional Neural Network (CNN) using TensorFlow and Keras. The work has four stages: load and preprocess the data, define the model, train the model, and estimate its performance.

Theory

Why CNN. A normal Dense network flattens an image and loses the position of pixels. It also needs a huge number of weights for color images. A CNN keeps the image structure and learns local patterns such as edges, corners and textures, using far fewer weights.

Parts of a CNN.

- **Convolution layer (Conv2D):** a small filter (here 3 × 3) slides over the image and produces a feature map that shows where a certain pattern appears. Many filters learn many different patterns. Early layers find edges, middle layers find shapes, and deeper layers find object parts.

- **Padding (same):** adds a border of zeros so the output image keeps the same size as the input.
- **ReLU activation:** removes negative values and adds non-linearity.
- **Pooling layer (MaxPooling2D):** takes the maximum value in each 2 × 2 block. It halves the image size, reduces computation, and keeps the strongest features.
- **Flatten:** converts the final feature maps into one long vector.
- **Dense layer:** combines all the learned features to decide the class.
- **Dropout (0.5):** randomly switches off half of the neurons during training. This prevents the model from depending on a few neurons and reduces overfitting.
- **Softmax output:** gives the probability of each of the 10 classes.

Dataset: CIFAR-10. It has 60,000 color images of size 32 × 32 in 10 classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship and truck. There are 50,000 training images and 10,000 test images. Each image has 3 channels (red, green, blue).

Optimizer: Adam. Adam adjusts the learning rate for each weight automatically. It usually trains faster and more smoothly than plain SGD.

Performance estimation.

- Test loss and test accuracy on unseen images.
- Predictions on sample images (predicted label compared with actual label).
- Learning curves: training vs validation accuracy and loss. If training accuracy keeps rising while validation accuracy stops, the model is overfitting.

Steps in Short

1. Load CIFAR-10, scale pixels to 0–1, one-hot encode the labels, and display sample images.
2. Build the model: three Conv + MaxPool blocks (32, 64, 128 filters), Flatten, Dense 128, Dropout 0.5, Dense 10 softmax.
3. Compile with Adam and categorical cross-entropy.
4. Train for 10 epochs with batch size 64 and a 10% validation split.
5. Evaluate on the test set, predict 10 images, and plot the accuracy and loss graphs.

Conclusion

A CNN image classifier was built using TensorFlow and Keras and trained on CIFAR-10. The images were normalized, the model was trained for 10 epochs, and its performance was measured using test loss, test accuracy, sample predictions and learning curves. The model reached roughly 70 percent test accuracy. Some mistakes happen between similar

classes such as cats and dogs. Accuracy can be improved with data augmentation, batch normalization, more epochs or a deeper network. The exact accuracy changes slightly in each run.

Practical 4: Anomaly Detection Using an Autoencoder

Aim

To implement an autoencoder for anomaly detection using TensorFlow and Keras: import the libraries, load the dataset, encode the input into a latent representation, decode it to reconstruct the input, and compile, train and evaluate the model with an optimizer, a loss function and evaluation metrics.

Theory

Anomaly detection. An anomaly is a sample that is very different from normal data. Examples are a fraud transaction, a faulty machine reading or a damaged product. Anomalies are rare, so it is hard to collect enough examples to train a normal classifier. An autoencoder solves this by learning only from normal data.

Autoencoder. An autoencoder is a neural network trained to copy its input to its output. It has two parts:

- **Encoder:** compresses the input into a small representation called the latent vector or bottleneck. Here the sizes are 784 → 128 → 64 → 32.
- **Decoder:** rebuilds the original input from the latent vector. Here the sizes are 32 → 64 → 128 → 784.

The bottleneck is small, so the network cannot simply copy the input. It is forced to keep only the most important features.

Main idea of the method.

1. Train the autoencoder only on normal data (digit 0).
2. The network becomes good at rebuilding normal data.
3. When an unseen different digit is given, the network rebuilds it badly.
4. The reconstruction error (difference between input and output) is therefore large for anomalies and small for normal samples.

Reconstruction error. It is the mean squared difference between the original image and the rebuilt image, calculated for each image. It works as the anomaly score.

Threshold. A value that separates normal from anomaly. Here it is the 95th percentile of the errors of normal training images. If an image's error is above the threshold, it is called an anomaly. About 5 percent of normal images will be wrongly flagged with this choice.

Functions used.

- **Sigmoid** in the last layer keeps the output pixels between 0 and 1, matching the normalized input.
- **MSE (mean squared error)** is the loss. **MAE (mean absolute error)** is an extra metric.
- **Adam** is the optimizer.

Evaluation terms.

- **True Positive (TP):** an anomaly correctly detected.
- **True Negative (TN):** a normal image correctly accepted.
- **False Positive (FP):** a normal image wrongly flagged as anomaly.
- **False Negative (FN):** an anomaly that was missed.
- **Accuracy:** (TP + TN) divided by all samples.
- **Precision:** TP divided by (TP + FP). Of all flagged images, how many were really anomalies.
- **Recall:** TP divided by (TP + FN). Of all real anomalies, how many were found.

Dataset: MNIST. Digit 0 images are the normal class and the other digits are the anomalies. MNIST is used only to demonstrate the idea.

Steps in Short

1. Load MNIST, normalize, and flatten each image to 784 values.
2. Keep only digit 0 images for training and make anomaly labels for the test set.
3. Build the encoder and decoder and join them into an autoencoder.
4. Compile with Adam and MSE, and train the model on normal images only.
5. Calculate the reconstruction errors and set the threshold using the 95th percentile.
6. Flag the images above the threshold and calculate accuracy, precision and recall.
7. Plot the training and validation loss.

Conclusion

An autoencoder with an encoder and a decoder was built and trained only on normal digit 0 images. The reconstruction error was used as an anomaly score, and a threshold taken from normal training errors was used to classify the test images. The model detected most of the non-zero digits as anomalies with high precision. Some anomalies that look like zero (for example 6 or 9) were missed, so recall was lower than precision. This is a

teaching demonstration. In real applications the normal and anomalous data must be defined for the problem, and the threshold should be chosen using a separate validation set.

Practical 5: Continuous Bag of Words (CBOW) Model

Aim

To implement a CBOW model that predicts a target word from its surrounding context words, in four stages: data preparation, generating the training data, training the model, and displaying the output.

Theory

Why word embeddings. Computers cannot understand words, only numbers. A simple way is one-hot encoding, but it has two problems: the vectors are very long for a big vocabulary, and they show no relationship between words (cat and dog look as different as cat and car). A word embedding gives each word a short vector of real numbers, and words used in similar places get similar vectors.

Word2Vec. Word2Vec is a family of methods that learn word embeddings from text. It has two models:

- **CBOW (Continuous Bag of Words):** predicts the target word from the surrounding context words.
- **Skip-gram:** predicts the surrounding words from the target word.

CBOW is faster and works well for frequent words. It is called a bag of words because the order of the context words is ignored.

Example. In the sentence "the cat sits on the mat", with a window size of 2, the target word "sits" has the context words [the, cat, on, the]. The model should learn to output "sits" for this context.

Window size. It is the number of words taken on each side of the target. Here it is 2, so up to 4 context words are used.

How the CBOW model works.

1. **Vocabulary:** all unique words are listed and given an ID number.
2. **Training pairs:** for each word, the context IDs are the input and the word's ID is the target.

3. **Padding:** words at the edges of a sentence have fewer than 4 context words, so zeros are added to make all inputs the same length. IDs are shifted by 1 so that 0 is used only for padding.
4. **Embedding layer:** converts each context word ID into a vector (here 16 numbers). These vectors are learned during training. Masking (mask_zero) makes the layer ignore the padding zeros.
5. **Average pooling:** the context vectors are averaged into one vector.
6. **Dense layer with softmax:** gives the probability of every word in the vocabulary being the target.

Loss function. Sparse categorical cross-entropy is used because the target is a single word ID and not a one-hot vector.

Limitation. The corpus here has only a few sentences, so the model is only a demonstration. Real systems train on millions of sentences.

Steps in Short

1. Prepare a small corpus, split it into words and build the vocabulary with word-to-ID and ID-to-word dictionaries.
2. Generate the context and target pairs using a window of 2.
3. Pad the contexts to length 4.
4. Build the model: Embedding, Average Pooling, Dense softmax.
5. Train with Adam for 100 epochs.
6. Predict the target word for a given context and plot the training loss.

Conclusion

A CBOW model was implemented on a small text corpus. Words were converted to IDs, training pairs were created from a context window, and an embedding-based network was trained to predict the target word. The training loss decreased over the epochs and the model predicted target words from the context. Because the corpus is very small, the predictions are only for understanding the CBOW workflow and are not meant for real language understanding.

Practical 6: Transfer Learning with a Pre-trained CNN (MobileNetV2)

Aim

To use transfer learning: load a pre-trained CNN, freeze its learned layers, add and train a custom classifier, and fine-tune selected layers of the network.

Theory

Transfer learning. In transfer learning, a model trained on one big task is reused for a new, related task. It is like hiring an experienced person and teaching only the new part of the job, instead of training a beginner from zero. The reason it works is that the early layers of a CNN learn general features (edges, colors, textures) that are useful for almost any image problem.

Advantages.

- Needs much less training data.
- Trains faster and needs less computing power.
- Usually gives better accuracy than training a small model from scratch.

Pre-trained model: MobileNetV2. It is a light and fast CNN trained on ImageNet, a dataset of over a million images in 1,000 classes. It is designed for mobile and low-power devices, so it runs quickly even on Colab.

Two main techniques.

1. **Feature extraction (freezing):** the pre-trained base is frozen (trainable = False) so its weights do not change. Only a new classifier added on top is trained. The base works as a fixed feature extractor.
2. **Fine-tuning:** after the classifier is trained, the top layers of the base are unfrozen and trained again with a very small learning rate (here 1e-5). This adapts the high-level features to the new dataset. A small learning rate is used so the useful pre-trained knowledge is not destroyed.

Why Batch Normalization layers stay frozen. Updating them with small data can make training unstable, so they are kept frozen during fine-tuning.

Other parts used.

- **include_top = False:** removes the original 1000-class classifier of ImageNet.
- **Resizing to 96 × 96:** CIFAR-10 images are only 32 × 32, and the pre-trained model works better with larger input.

- **preprocess_input:** scales the pixels in the way MobileNetV2 was originally trained (to the range -1 to 1).
- **GlobalAveragePooling2D:** averages each feature map into one number, giving a compact vector.
- **Dense + Dropout + Softmax:** the new classifier for the 10 classes.

Note on object detection. CIFAR-10 has only one label per image and no bounding boxes. This practical therefore shows transfer learning for image classification, which is a basic building block of object detection. Real object detection needs bounding-box data and models such as SSD, Faster R-CNN or YOLO.

Steps in Short

1. Load a subset of CIFAR-10 (10,000 training and 2,000 test images).
2. Load MobileNetV2 with ImageNet weights, without its top, and freeze it.
3. Add resizing, preprocessing, pooling and a new classifier.
4. Train only the new classifier for 5 epochs.
5. Unfreeze the top 20 percent of the base, keep batch normalization frozen, recompile with a learning rate of 1e-5, and train for 3 more epochs.
6. Evaluate on test data and plot accuracy, with a dashed line where fine-tuning starts.

Conclusion

MobileNetV2 pre-trained on ImageNet was used as a feature extractor. Its layers were first frozen while a custom classifier was trained, and then the upper layers were unfrozen and fine-tuned with a lower learning rate. The model reached about 75 percent test accuracy using only a small part of the data and a few epochs, which is better than the CNN trained from scratch in Practical 3. This shows that transfer learning saves time and data. The practical demonstrates classification, and real object detection would additionally need bounding-box annotations and a detection architecture.

Quick Comparison of All Practicals

- **Practical 2:** FFNN on MNIST. Supervised classification with Dense layers. About 90 percent accuracy.
- **Practical 3:** CNN on CIFAR-10. Supervised classification with convolution layers. About 70 percent accuracy.
- **Practical 4:** Autoencoder on MNIST. Unsupervised anomaly detection using reconstruction error.

- **Practical 5:** CBOW on a small text corpus. Learns word embeddings by predicting a word from its context.
- **Practical 6:** MobileNetV2 on CIFAR-10. Transfer learning with freezing and fine-tuning. About 75 percent accuracy.